

atomic_accessor

Document #: P2689
Date: 2022-10-14
Project: Programming Language C++
LEWGI and SG1
Reply-to: Christian Trott
<crtrott@sandia.gov>
Damien Lebrun-Grandie
<lebrungrandt@ornl.gov>
Mark Hoemmen
<mhoemmen@nvidia.com>
Daniel Sunderland
<dansunderland@gmail.com>

Contents

- [1 Revision History](#)
 - [1.1 Initial Version 2022-10 Mailing](#)
- [2 Rational](#)
- [3 atomic_accessor implementation](#)
- [4 Open Question: relaxed atomic operations](#)
- [5 Wording](#)
- [6 Acknowledgements](#)

§ 1 Revision History

§ 1.1 Initial Version 2022-10 Mailing

§ 2 Rational

This proposal adds an `atomic_accessor` to the C++ standard, to be used with `mdspan`. When accessing data through an `mdspan` with an `atomic_accessor` all operations are performed atomically, by using `atomic_ref` as the `reference_type`. `atomic_accessor` was part of the rational provided in P0009 for `mdspans` *accessor policy* template parameter.

One of the primary use case for this accessor is the ability to write algorithms with somewhat generic mdspan outputs, which can be called in sequential and parallel contexts. When called in parallel contexts users would simply pass an mdspan with an `atomic_accessor` - the algorithm implementation itself could be agnostic to the calling context.

A variation on this use case is an implementation of an algorithm taking an execution policy, which adds the `atomic_accessor` to its output argument if called with a parallel policy, while using the `default_accessor` when called with a sequential policy. The following demonstrates this with a function computing a histogram:

```
template<class T, class Extents, class LayoutPolicy>
auto add_atomic_accessor_if_needed(
    std::execution::sequenced_policy,
    mdspan<T, Extents, LayoutPolicy> m) {
    return m;
}

template<class ExecutionPolicy, class T, class Extents, class
    LayoutPolicy>
auto add_atomic_accessor_if_needed(
    ExecutionPolicy,
    mdspan<T, Extents, LayoutPolicy> m) {
    return mdspan(m.data_handle(), m.mapping(), atomic_accessor<T>());
}

template<class ExecT>
void compute_histogram(ExecT exec, float bin_size,
    mdspan<int, stdex::dextents<int,1>> output,
    mdspan<float, stdex::dextents<int,1>> data) {

    static_assert(is_execution_policy_v<ExecT>);

    auto accumulator = add_atomic_accessor_if_needed(exec, output);

    for_each(exec, data.data_handle(), data.data_handle()+data.extent(0),
        [=](float val) {
            int bin = std::abs(val)/bin_size;
            if(bin > output.extent(0)) bin = output.extent(0)-1;
            accumulator[bin]++;
        });
}
```

The above example is available on godbolt: <https://godbolt.org/z/jY17Yoje1> .

§ 3 atomic_accessor implementation

The implementaiton of an `atomic_accessor` is straightforward when leveraging `atomic_ref`:

```
template <class ElementType>
struct atomic_accessor {
```

```

using offset_policy = atomic_accessor;
using element_type = ElementType;
using reference = std::atomic_ref<element_type>;
using data_handle_type = ElementType*;

constexpr atomic_accessor() noexcept = default;

template<class OtherElementType>
requires (
    std::is_convertible_v<OtherElementType(*)[], element_type(*)[]>
)
constexpr atomic_accessor(stdex::default_accessor<OtherElementType>)
    noexcept {}

template<class OtherElementType>
requires (
    std::is_convertible_v<OtherElementType(*)[], element_type(*)[]>
)
constexpr atomic_accessor(atomic_accessor<OtherElementType>) noexcept {}

constexpr data_handle_type
offset(data_handle_type p, size_t i) const noexcept {
    return p + i;
}

constexpr reference access(data_handle_type p, size_t i) const noexcept
{
    return reference(p[i]);
}
};

```

Compared to `default_accessor` we simply make the reference an `atomic_ref<element_type>`. We also add a conversion from `default_accessor` which would allow simple conversion of `mdspans` from non-atomic to atomic:

```

using array_t = mdspan<int, dextents<int, 2>>;
using atomic_array_t = mdspan<int, dextents<int, 2>,
    layout_right, atomic_accessor<int>>;

// Assigning from non-atomic to atomic is fine:
void foo(array_t a) {
    atomic_array_t atomic_a = a;
    ...
}

```

§ 4 Open Question: relaxed atomic operations

In the majority of use cases for using atomic accesses on multi-dimensional arrays, those atomics are used to deal with simple accumulations in the presence of concurrent updates.

The previous example demonstrates how it is possible to write generic algorithms which can be used in concurrent and non-concurrent situations.

However: in most of these cases relaxed atomics are all that is needed. Using sequentially consistent atomics instead comes with a significant performance penalty on architectures with more relaxed memory semantics than X86. However: the simple operators (such as `operator+=`) of `atomic_ref` are doing sequentially consistent operations.

On the `atomic_accessor` side this could be accounted for with an additional template argument taking the `memory_order`.

That leaves the question of the reference type for such a `memory_order` aware accessor, with three options coming immediately to mind:

1. add a template parameter to `atomic_ref` which defaults to `memory_order_seq_cst`.
2. add a new class `relaxed_atomic_ref` which has the same implementation as `atomic_ref` with the difference that the default `memory_order` of its update functions (e.g. `fetch_add`) is `memory_order_relaxed`.
3. have `relaxed_atomic_ref` as an exposition only class for `atomic_accessor`

While we believe that option 1. would be preferable we recognize that it is a breaking change, and thus unlikely to be acceptable to the committee. Thus we would like to champion option 2, with the possible implication that we should introduce both `atomic_accessor` and `relaxed_atomic_accessor` to correspond to the two classes.

A natural question arises regarding other memory orders. We do not believe that there are many use cases requiring anything other than relaxed or sequentially consistent atomic operations on multidimensional arrays. It is unlikely that multidimensional arrays will be widely used to implement complicated synchronization mechanism.

§ 5 Wording

Wording will be provided after an initial review, and initial guidance on how to deal with the desire for easy relaxed atomic operations.

§ 6 Acknowledgements

Sandia National Laboratories is a multimission laboratory managed and operated by National Technology and Engineering Solutions of Sandia, LLC., a wholly owned subsidiary of Honeywell International, Inc., for the U.S. Department of Energy's National Nuclear Security Administration under Grant DE-NA-0003525.

This manuscript has been authored by UTBattelle, LLC, under Grant DE-AC05-00OR22725 with the U.S. Department of Energy (DOE).

This work was supported by Exascale Computing Project 17-SC-20-SC, a joint project of the U.S. Department of Energy's Office of Science and National Nuclear Security Administration, responsible for delivering a capable exascale ecosystem, including software, applications, and hardware technology, to support the nation's exascale computing imperative.